

HOWTO: rcs/nighthawk_qrack.py

A clean-qubit PyQrack companion to arXiv:2609.28657 (random-circuit sampling on IBM Nighthawk r2)

Script	rcs/nighthawk_qrack.py (ThereminQ examples)
Companion data	BlueQubitDev/rcs-nighthawk
Paper under study	Sedrakyan et al., arXiv:2609.28657 (preprint, v1)
Simulator	Qrack / PyQrack — QrackSimulator , QrackAceBackend
Acceleration	OpenCL or CPU. No CUDA.

1. Purpose and scope

arXiv:2609.28657 reports forward random-circuit sampling (RCS) on 61 qubits of the 120-qubit IBM Nighthawk r2 processor (`ibm_phoenix`), with native CZ gates on a square lattice, and estimates the circuit fidelity at 36 cycles from two proxy families: *patched* circuits scored with a normalised linear cross-entropy benchmark (XEB), and *mirror* circuits scored by echo survival. BlueQubit released the circuits, layout, partitions and measured bitstrings as `rcs-nighthawk`.

`nighthawk_qrack.py` does four things against that release, without a QPU:

1. **Regenerates** every released circuit bit-for-bit from `data/layout.json` and checks it against the released OpenQASM (`verify`).
2. **Re-scores** the measured `ibm_phoenix` bitstrings with PyQrack ideal patch distributions and rebuilds the paper's F(d) points, exponential fit and collision ratios (`hwxeb`).
3. **Emulates** the released circuits on ideal ("clean") qubits, scores them with the same estimators, and sets the simulator's per-cycle fidelity next to the device's (`run`). With `--backend ace` the only error left is shot noise plus `QrackAceBackend`'s own seam approximation; with `--backend exact` the result is a pipeline control that must give $F = 1$.
4. Runs a **data-only** analysis of the release (`seamgap`): CZ counts per circuit family, the fit read at depth versus at gate count, and a per-cycle error budget.

There is **no noise model** in the current version. Any deviation from $F = 1$ under `run` is attributable to the simulator backend or to sampling, not to an assumed device error.

Version note. This HOWTO describes the script on `main` as fetched from `raw.githubusercontent.com` (1111 lines; docstring "*clean-qubit PyQrack companion to arXiv:2609.28657*"). GitHub's rendered blob view may briefly show an earlier revision that contained a noise-model audit (`--preset r2`, `--stochastic`, `--edge-layers`); those flags no longer exist.

2. Background

2.1 Random-circuit sampling and linear XEB

Each cycle applies a Haar-random single-qubit gate to every qubit, then a layer of CZ gates on one of four coupler colourings `A`, `B`, `C`, `D` in fixed order. The linear XEB was introduced as a fidelity proxy for such circuits by Boixo et al. [2] and used for the 53-qubit Sycamore experiment [3]. Its relationship to the true fidelity, and the phases in which it is and is not a reliable estimator, are analysed in [4] and [5].

2.2 Patched circuits (Eq. (1)-(2) of the paper)

A patched circuit removes every CZ that crosses a partition boundary, so the 61-qubit register factorises into K independent patches whose ideal output distributions p_r are exactly computable. The estimator is the product over patches of the normalised patch XEB:

$$F_{\text{patched}} = \prod_r \text{mean}_s \left[\left(2^{\{n_r\}} \cdot p_r(x_r^{\{s\}}) - 1 \right) / \left(2^{\{n_r\}} \cdot \sum_x p_r(x)^2 - 1 \right) \right]$$

where $x_r^{\{s\}}$ is the restriction of shot s to patch r . The standard error uses a second-order delta method on the product of means (`_delta_se`). Patch circuits as a fidelity proxy trace back to the "patch" and "elided" circuits of [3].

2.3 Mirror circuits

A mirror circuit runs U for $d/2$ cycles and then $U^\dagger$ for $d/2$ cycles, so the ideal output is the input string; the fidelity proxy is the survival probability. This construction is the "circuit mirroring" of Proctor et al. [6], developed into scalable randomized benchmarking in [7] and extended to universal gate sets in [8].

In the release, the forward half is **pseudo-patched**: at each cycle the CZs crossing *one* of the five K=3 boundary sets are removed, rotating to the next partition every cycle (`rotate_every=1`). The script reproduces this exactly (`mirror_cycles`).

2.4 Pauli-frame twirling

Optional gate twirling surrounds each CZ with random Paulis $P_a \otimes P_b$ before and the propagated correction $(P_a Z^{[P_b \in \{X,Y\}]}) \otimes (P_b Z^{[P_a \in \{X,Y\}]})$ after, which is an exact identity for ideal gates (`cz_layer`). This is the randomized-compiling construction of Wallman and Emerson [9]. On the ACE backend its role is to randomise ACE's *coherent* seam error so that the forward and inverse halves of a mirror cannot cancel it. On the exact backend twirling is disabled because it cannot change the result.

2.5 Qrack backends

- exact** — `QrackSimulator`. Patched circuits factorise and Qrack's QUnit layer keeps the patches separable, so 61-qubit patched circuits run exactly. Mirror circuits entangle all 61 qubits after one ABCD sweep; use `--n / --sizes` to truncate to the first n logical qubits (the release's own `e < n` convention).
- ace** — `QrackAceBackend` with `noise=0`, an approximate backend that splits the register at "seams" controlled by `long_range_columns / long_range_rows` (`--lrc, --lrr`). The 61 logical qubits are placed at their true device positions on an 8×8 ACE grid (device rows 1-8, cols 2-9), so every coupler is nearest-neighbour in ACE's grid; the three unused sites idle in |0>. `is_torus=False`.

Qrack's exact and approximate simulation methods are described in Strano et al. [10].

3. Requirements

Component	Notes
Python	≥ 3.9 (uses <code>argparse.BooleanOptionalAction</code>)
numpy	required by every subcommand
PyQrack	required by <code>hwxeb, run, selftest</code> ; not by <code>verify</code> or <code>seamgap</code> . The <code>ace</code> backend needs a PyQrack build that ships <code>QrackAceBackend</code> .
Qrack shared library	if <code>/usr/local/lib/qrack/libqrack_pinvoke.so</code> exists, the script exports it as <code>PYQRACK_SHARED_LIB_PATH</code> automatically
Qiskit	not required
GPU	OpenCL (select with <code>QRACK_OCL_DEFAULT_DEVICE</code>) or pass <code>--cpu</code> for <code>is_gpu=False</code> . No CUDA path is used.

The script's header block, which every PyQrack script in this repository carries:

```
QRACK_LIB_PATH = "/usr/local/lib/qrack/libqrack_pinvoke.so"
if os.path.exists(QRACK_LIB_PATH):
    os.environ["PYQRACK_SHARED_LIB_PATH"] = QRACK_LIB_PATH
```

3.1 Environment variables

Variable	Effect
<code>QRACK_OCL_DEFAULT_DEVICE</code>	OpenCL device index when <code>--cpu</code> is not given
<code>QRACK_MAX_CPU_QB</code>	upper bound on qubits for CPU state vectors; raise it for large <code>--backend exact mirror</code> truncations
<code>QRACK_MAX_ALLOC_MB</code>	Qrack's memory ceiling for large exact states

If the exact engine cannot be built for a given n (e.g. above `QRACK_MAX_CPU_QB`), `run` prints a note and skips that size rather than aborting.

4. Installation

```
# 1. Script
git clone https://github.com/twobombs/thereminq-examples
cd thereminq-examples/rcs

# 2. Released circuits, layout and hardware counts
git clone https://github.com/BlueQubitDev/rcs-nighthawk

# 3. Python dependencies (inside the ThereminQ container these are already present)
pip install numpy pyqrack
```

`--repo` points at the `rcs-nighthawk` checkout for every subcommand. The script reads, relative to it:

Path	Used by
data/layout.json	all (qubit map, matchings, schedule, partitions, seeds, depths, mirror input strings)
data/circuits/manifest.json, data/circuits/**/*.qasm	verify, selftest, seamgap
data/counts/patched_K{K}_d{d}.npz, data/counts/mirror_survival.json	hwxeb, selftest
data/results/patch_xeb.json, data/results/fidelity_vs_depth.json	hwxeb, run --summarize, selftest, seamgap

5. Recommended workflow

Run the steps in this order; each one validates an assumption the next one relies on.

```

# Step 1 – circuits are regenerated exactly (no Qrack needed)
python nighthawk_qrack.py verify --repo rcs-nighthawk

# Step 2 – conventions and exactness checks (seconds)
python nighthawk_qrack.py selftest --repo rcs-nighthawk

# Step 3 – the paper's hardware numbers, re-scored with PyQrack as reference
python nighthawk_qrack.py hwxeb --repo rcs-nighthawk --cpu --cache patch_cache

# Step 4 – clean-qubit emulation on ACE, mirror family, full 61-qubit register
python nighthawk_qrack.py run --repo rcs-nighthawk --backend ace --families mirror \
--depths 4 8 16 24 36 --out clean_ace.jsonl

# Step 5 – size sweep to fit b(N) and extrapolate to N = 61
python nighthawk_qrack.py run --repo rcs-nighthawk --backend ace --families mirror \
--sizes 27-36,61 --depths 4 6 8 12 --out clean_ace.jsonl

# Step 6 – exact pipeline control on the patched family
python nighthawk_qrack.py run --repo rcs-nighthawk --backend exact --families patched \
--depths 20 36 --out clean_exact.jsonl

# Step 7 – tables only, from what is already on disk
python nighthawk_qrack.py run --repo rcs-nighthawk --summarize --out clean_ace.jsonl

```

6. Subcommand reference

6.1 verify — circuit regeneration

Rebuilds every released circuit from layout.json exactly as rcs/circuits.py does and compares it gate-by-gate with the released QASM.

- **Angle generator:** SplitMix64 hash of (seed, instance, cycle, qubit, k) → Haar angles (φ, θ, λ), applied as $r_z(\varphi) \cdot r_x(\theta) \cdot r_z(\lambda)$.
- **Seeds:** $2025 + k \cdot 1000003$ for full and patched circuits; 2025 with instance index k for mirror circuits.
- **Families checked:** mirror (forward + exact inverse), patched $K=3$ and $K=4$ over all five partitions and instances, full circuits at depths 4, 8, ..., 40.
- **Checks:** operation sequence, qubit indices, angles modulo 2π within --tol (default $1e-9$ rad), and CZ count against manifest.json.

Flag	Default	Meaning
--repo	.	rcs-nighthawk checkout
--tol	1e-9	angle tolerance, rad

Exit status is 0 only if every circuit is identical and none in the manifest was left unregenerated.

6.2 selftest — conventions and exactness

#	Check	Pass criterion
1	angle hash + pseudo-patch rotation vs <code>mirror/d08_instance1.qasm</code>	identical
2	<code>u_angles</code> → Qrack <code>u</code> equals the Haar matrix	overlap within 1e-5 of 1
3	exact mirror, first <code>--n</code> logical qubits, <code>d = 12</code> , all 10 inputs	$ 1 - F < 1e-4$
4	Pauli-frame twirl is an identity (exact backend)	$ 1 - F < 1e-4$
5	PyQrack patch ideal XEB vs the release's values (Aer, double)	max diff < 1e-3
6	Eq. (1)-(2) on the hardware counts vs the release	within 0.05σ
7	ideal samples score $F = 1$	within 4σ
8	ACE register geometry	reports grid size and that all couplers are nearest-neighbour

Flag	Default	Meaning
<code>--repo</code>	<code>.</code>	data checkout
<code>--n</code>	14	register size for the exact mirror checks
<code>--lrc</code> , <code>--lrr</code>	4, 4	ACE parameters for check 8

6.3 hwxeb — the paper's numbers with PyQrack as reference

For each $K \in \text{--K}$ and depth, loads the measured `ibm_phoenix` bitstrings, computes every patch's ideal distribution with `QrackSimulator` (one small simulator per patch, renormalised in float64), and applies Eq. (1)-(2). Per (K, d) it prints the inverse-variance-weighted fidelity, the release's value, and the mean collision ratio (patch ideal XEB) next to the release's. It then refits the mirror survival data from `mirror_survival.json` with the paper's unweighted log10 exponential fit and reports $F(36)$ and error per qubit per cycle.

The closing line reports the worst deviation from the release in units of σ and the worst ideal-XEB difference; PyQrack's float32 wheels are expected to agree to about 1e-5.

Flag	Default	Meaning
<code>--K</code>	3 4	partition sizes
<code>--depths</code>	release's patched depths	subset of depths
<code>--limit</code>	0	stop after this many circuits (0 = all 180)
<code>--cache</code>	none	directory for ideal patch distributions (float32 <code>.npz</code>); strongly recommended, also used by <code>run</code>
<code>--cpu</code>	off	<code>is_gpu=False</code>
<code>--out</code>	none	write per-circuit records in <code>patch_xeb.json</code> format

6.4 run — clean-qubit emulation

Executes the released circuits on ideal qubits and scores them with the paper's estimators.

Families (`--families`, comma list):

Family	What runs	Score
<code>mirror</code>	pseudo-patched $U(d/2)$ then exact inverse, over the ten released input strings, shots split evenly across inputs (and across twirls on ACE)	survival probability, binomial σ
<code>patched</code>	K-patch circuit on the 61-qubit register (only at <code>n = 61</code>)	Eq. (1)-(2), delta-method σ
<code>full</code>	unpatched forward circuit, release seed, instance 0; <code>--pubs</code> pubs of 100 000 shots	none — stored only, not verifiable at 61 qubits

Flags:

Flag	Default	Meaning
--backend	ace	exact or ace
--families	mirror,patched	see above
--depths	release's depths per family	cycles
--K	3 4	patched partition sizes
--instances	3 (from layout)	circuit instances
--partitions	all 5	use the first j partitions
--shots	paper	paper = Appendix-D budgets (below), or an integer per circuit / pub
--twirls	64	ACE mirror Pauli-frame randomisations
--exact-probs / --no-exact-probs	on	exact mirror: read survival with prob_perm (no bitstrings) or sample and store them
--sizes	none	register sizes, e.g. 36, 27-36, 20,27-36; first-n truncation (mirror); patched only at 61
--n	none	single size; mutually exclusive with --sizes
--pubs	10	full family pubs
--lrc, --lrr	4, 4	ACE long_range_columns, long_range_rows
--cache	none	ideal patch distributions
--shots-dir	<out stem>_shots/	bitstring root
--pack	off	only rebuild release-format files from stored points
--cpu	off	is_gpu=False
--out	nighthawk_clean.jsonl	results log
--summarize	off	print tables from --out and exit

Paper shot budgets (--shots paper):

Depth	4-8	10-14	16-18	20	24	28-32	36	40
mirror	30 000	45 000	60 000	72 000	96 000	120 000	240 000	360 000
patched	—	—	—	2 400	4 800	12 000 / 24 000	36 000	54 000

Resumability. Every configuration is tagged <backend>-<sha1[:8]> over backend, shots, twirls, ACE parameters and exact-probs mode — but *not* the register size, so a sweep can be extended with more sizes and resumes per (n, point). Records with a different tag in the same JSONL are ignored. Bitstrings are written atomically *before* the JSONL record, so an interrupted run never leaves a record without its data.

6.5 seamgap — data-only analysis

NumPy only. Prints: CZ counts (mean and range) per family and depth with the gap between the sampled full circuit and the mirror / K3 proxies; the paper's fit evaluated at d = 36 and at the gate-count-equivalent depth; the fit corrected for the missing CZs at three Pauli error rates; a weighted regression $\ln F = a + \beta \cdot \text{cycles} + \gamma \cdot \text{CZ}$ (common intercept and per-family offsets, depths $\geq$ --dmin, default 20); the K=3 vs K=4 ratio and its implied ϵ_{CZ} ; and a per-cycle budget of fitted decay vs CZ and single-qubit contributions (RB rates converted to Pauli rates by (d+1)/d).

7. Outputs

7.1 JSONL record (run)

One line per point:

```
{ "cfg": "ace-1a2b3c4d", "n": 61, "family": "mirror", "K": 0, "depth": 16,
  "partition": 0, "instance": 1, "seconds": 42.1,
  "fidelity": 0.97, "se": 0.0007, "hits": [...], "shots": 60000,
  "shots_per_input": [...], "shots_file": "..._shots/ace-1a2b3c4d/points/n61/mirror_K0_d16_p0_i1.npz" }
```

Patched records additionally carry patch_fidelity and patch_ideal_xeb.

7.2 Bitstrings in the release's own layout

After each run, per-point files under <shots-dir>/<cfg>/points/n{n}/ are merged into <shots-dir>/<cfg>/release/ so BlueQubit's analysis scripts read them unchanged:

File	Contents
counts/patched_K{K}_d{d}.npz	keys partition{j}_instance{i}, sorted uint64 shots
counts/mirror_survival.json	hits[i][s] of shots[i][s] per instance and input string
counts/mirror_shots_d{dd}.npz	every raw mirror shot and its twirl index
samples/full_d{d}.npz	shots, sorted

Truncated registers get an `_n{n}` suffix. Bit `q` is logical qubit `q` throughout (little-endian).

7.3 Summary tables (run `--summarize`)

- Per `n` and family: $F_{\text{sim}} \pm \sigma$; at `n = 61` also F_{hardware} and $F_{\text{sim}} / F_{\text{hw}}$.
- Mirror fit $F(d) = A \cdot f^d$ for the simulator, beside the device's fit at `n = 61`.
- Per-cycle decay $\mathbf{b(N)} = -\ln \mathbf{f}$, CZ per cycle, and error per qubit per cycle. Depths whose F falls at or below `3/shots` are dropped from the fit and marked `*`.
- With ≥ 2 sizes: a non-negative fit $\mathbf{b} = \mathbf{u} \cdot \mathbf{N} + \mathbf{v} \cdot \mathbf{CZ/cycle}$, evaluated at `N = 61`, and the ratio of simulator to device decay. `N` and `CZ/cycle` are nearly collinear; the script itself advises trusting `b(61)`, not `u` and `v` separately, and flags sweeps with fewer than five sizes or a span under six qubits as indicative only.

8. Interpreting the results

Backend / family	Expected	What a deviation means
exact, any	$F = 1$ ($b = 0$) up to shot noise	a pipeline or convention error; stop and run <code>selftest</code>
ace, mirror	$F < 1$, decaying with depth	ACE's seam (elision) error per cycle for the chosen <code>--lrc/--lrr</code> ; compare <code>b_ace</code> to the device's <code>b</code>
ace, patched	≤ 1	patched circuits have no cross-partition CZs, but ACE's seams are set by <code>--lrc/--lrr</code> , not by the partitions, so ACE error can still enter
hwxeb	matches release to $\ll 1 \sigma$	Qrack and the release's reference simulator agree on the ideal distributions

A clean-qubit simulator whose mirror decay is *slower* than the device's is a lower bound on the approximation cost of that backend, not a spoofing result: the estimators here are the paper's own proxies, whose limits as certificates of fidelity are discussed in [4] and [5].

9. Limitations

- The `full` family cannot be scored at 61 qubits; samples are stored for downstream use only.
- `--backend exact mirror` at `n = 61` is infeasible; use first-`n` truncation.
- Patched circuits use the released 61-qubit partitions and run only at `n = 61`.
- `QrackAceBackend` is an approximate method; results depend on the PyQrack version and the `--lrc/--lrr` setting. Record both with every published number.
- arXiv:2609.28657 is a preprint (v1) and had not been peer reviewed at the time of writing.

10. References

All arXiv identifiers below were checked against the arXiv abstract pages or their published versions.

[1] T. Sedrakyan *et al.*, "Quantum computational advantage in random-circuit sampling on IBM superconducting quantum computers," [arXiv:2609.28657](#) (2026). — *The experiment, estimators (Eq. (1)–(2)), partitions (App. B), pseudo-patching (App. C), shot budgets (App. D)*.

[2] S. Boixo *et al.*, "Characterizing Quantum Supremacy in Near-Term Devices," [arXiv:1608.00263](#); *Nature Physics* **14**, 595 (2018). — *Cross-entropy benchmarking as a fidelity proxy*.

[3] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," [arXiv:1910.11333](#); *Nature* **574**, 505 (2019). — *Linear XEB on 53 qubits; patch and elided verification circuits*.

[4] A. Morvan *et al.* (Google Quantum AI), "Phase transition in Random Circuit Sampling," [arXiv:2304.11119](#); *Nature* **634**, 328 (2024). — *Regimes in which XEB tracks fidelity; weak-link model*.

[5] X. Gao *et al.*, "Limitations of Linear Cross-Entropy as a Measure for Quantum Advantage," [arXiv:2112.01657](#); *PRX*

Quantum **5**, 010334 (2024). — *When XEB and fidelity diverge; spoofing by cutting weak links.*

[6] T. Proctor, K. Rudinger, K. Young, E. Nielsen, R. Blume-Kohout, "Measuring the Capabilities of Quantum Computers," [arXiv:2008.11294](#). — *Circuit mirroring*.

[7] T. Proctor *et al.*, "Scalable randomized benchmarking of quantum computers using mirror circuits," [arXiv:2112.09853](#). — *Mirror-circuit RB*.

[8] J. Hines *et al.*, "Demonstrating Scalable Randomized Benchmarking of Universal Gate Sets," [arXiv:2207.07272](#); *Phys. Rev. X* **13**, 041030 (2023). — *Mirror RB with continuously parametrised (Haar-type) single-qubit gates*.

[9] J. J. Wallman, J. Emerson, "Noise tailoring for scalable quantum computation via randomized compiling," [arXiv:1512.01098](#); *Phys. Rev. A* **94**, 052325 (2016). — *Pauli-frame twirling*.

[10] D. Strano *et al.*, "Exact and approximate simulation of large quantum circuits on a single GPU," [arXiv:2304.14969](#); IEEE QCE 2023. — *Qrack's exact and approximate simulation methods*.

Software and data (not on arXiv): [BlueQubitDev/rcs-nighthawk](#) · [unitaryfund/pyqrack](#) · [twobombs/thereminq-examples](#)